**Tuwaiq Academy**
AI and Data Science Diploma

# GrowthLens

## Predicting Three-Year Company Market-Cap Growth

Machine Learning Project Report

| | |
|---|---|
| **Student Name** | Yasser Al-Tayari |
| **Instructor** | Banan Alnemri |
| **Course** | Scalable Data Science with Python |
| **Submission Date** | [13/08/2026] |

Large-Scale GPU-Accelerated Machine Learning Project

# Contents

# 1 Abstract

GrowthLens is a GPU-accelerated machine learning project designed to publicly traded small and medium-sized companies with strong future growth potential. The project uses a dataset containing 1,000,000 company-date observations, 48 original features, and 2,454 U.S. companies observed between April 2021 and July 2023. A binary target was created by classifying an observations as an Opportunity when its market capitalization increased by more than 50% over the following three years. RAPIDS cuDF was used for large-scale data loading and manipulation, while GPU-accelerated CatBoost, XGBoost, and cuML models were evaluated. Chronological splitting was applied to assess how well the models generalize to newer markets periods. CatBoost achieved the strongest performance among the long-term chronological experiments, with an AUC of 0.7398, accuracy of 0.7011, and F1 score of 0.5343. An additional three-month/one-month XGBoost experiment achieved an AUC of 0.9632, but its results were treated cautiously because observations from the same companies and overlapping target periods may appear across folds. The results demonstrate that company growth opportunities contain useful predictive patterns, but temporal market drift remains a major challenge.

# 2 Introduction

## 2.1 Background and Context

Identifying companies with strong future growth potential is an important problem for investors, acquisition teams, and corporate decision-makers. Financial decisions are commonly based on company fundamentals, market performance, liquidity, profitability, and industry conditions. However, manually analyzing thousands of companies and repeated historical observations is time-consuming and may cause promising companies to be overlooked.

Machine learning can analyze large volumes of historical company information and identify patterns associated with future market-cap growth. GPU acceleration also makes it possible to process and model millions of observations more efficiently than conventional sequential workflows

## 2.2 Project Motivation

GrowthLens was developed as a decision-support system for identifying smaller publicly traded companies that may experience substantial future growth. The system does not replace financial due diligence or guarantee investment success. instead, it provides an initial screening tool that helps decision-makers reduce large company universe into smaller list of candidates for further investigations.

# 3 Problem Statement

The project addresses the difficulty of identifying companies that may significantly increase their market capitalization in the future. A company can currently have a relatively low market value while still possessing characteristics associated with substantial future growth.

Each observation contains company information available at a specific date, including market momentum, volatility,liquidity, profitability, leverage, financial growth, sector and industry. The output is a binary classification representing whether the company subsequently achieved more than 50% market-cap growth over approximately three years.

# 4 Objectives

- Process and analyze a large-scale financial dataset using RAPIDS cuDF.
- Predict three-year company market-cap growth using machine learning.
- Compare at least three machine learning algorithms.
- evaluate model performance using Accuracy, F1-score,Precision, and Recall.
- Measure GPU utilization, memory usage, training time, and inference speed.
- Identify the factors that contribute most strongly to the predictions.

# 5 Data

## 5.1 Data Source

The structured dataset was created by combining historical market information obtained through the Alpaca Market Data API with company financial statement and filling information published through the U.S. Securities and Exchange Commission EDGAR system. Company identifiers and metadata corresponding public companies. The final dataset was stored in parquet format because provides efficient columnar storage and can be loaded directly into GPU memory using cuDF.

## 5.2 Dataset Description

The dataset contains 1,00,000 rows and 48 columns representing 2,454 publicly traded U.S. companies. The observations cover the period from 30 April 2021 to 27 July 2023. Each row represent one company at one observation date. Therefore, the same company can appear in several rows at different dates. Companies contain an average of 407.5 observations, with a maximum of 542 observations.

Table 1: Dataset overview

| **Property** | **Value** |
|---|---|
| Number of rows | 1,000,000 |
| Number of original columns | 48 |
| Machine learning task | Classification |
| Primary target | target_marketcap_growth_3y_pct |
| Training rows | 793,277 |
| Validation rows | 101,028 |
| Test rows | 105,695 |

## 5.3 Target Variable

The primary target is `target_marketcap_growth_3y_pct`, representing the percentage change in a company's market capitalization approximately three years after the observation date.

$$\text{Growth}_{3y}(\%) = \left( \frac{\text{Future Market Cap}}{\text{Current Market Cap}} - 1 \right) \times 100 \tag{1}$$

## 5.4 Exploratory Data Analysis

Exploratory data analysis was conducted to examine the target distribution, temporal changes, numerical relationships, and differences between business sectors.


Figure 1: Distribution of the binary target classes.

**Interpretation.** The dataset contains 655,812 Not Opportunity observations and 344,188 Opportunity observations. The positive class represents 34.42% of the dataset, indicating moderate class imbalance.

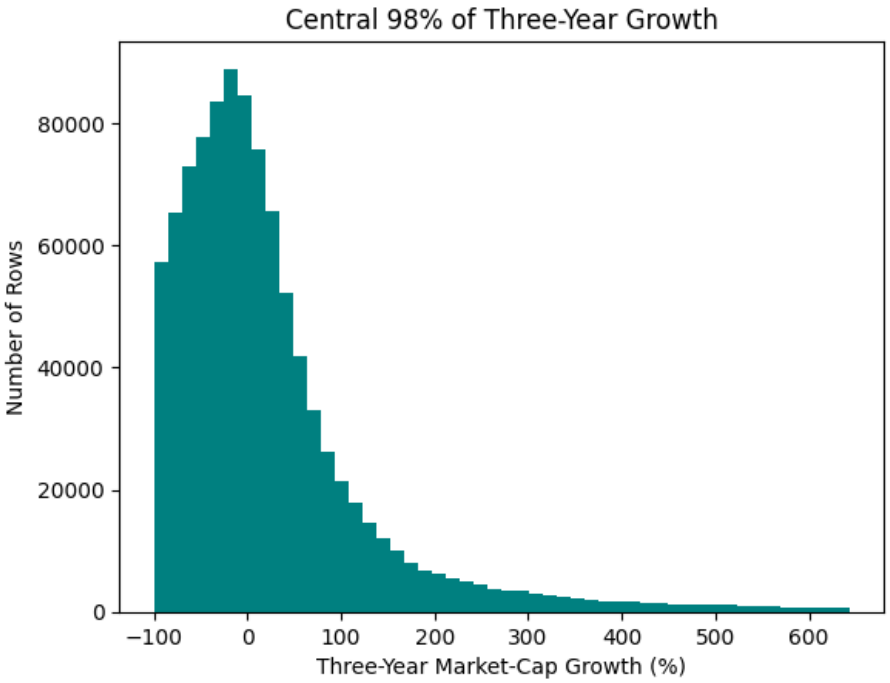



Figure 2: Central 98% of three-year market-cap growth.

**Interpretation.** The original growth target is strongly right-skewed. Most observations experienced losses or moderate growth, while a smaller number achieved very high positive growth. The upper 2% was excluded only from this visualization to improve readability and was not removed from the classification dataset.

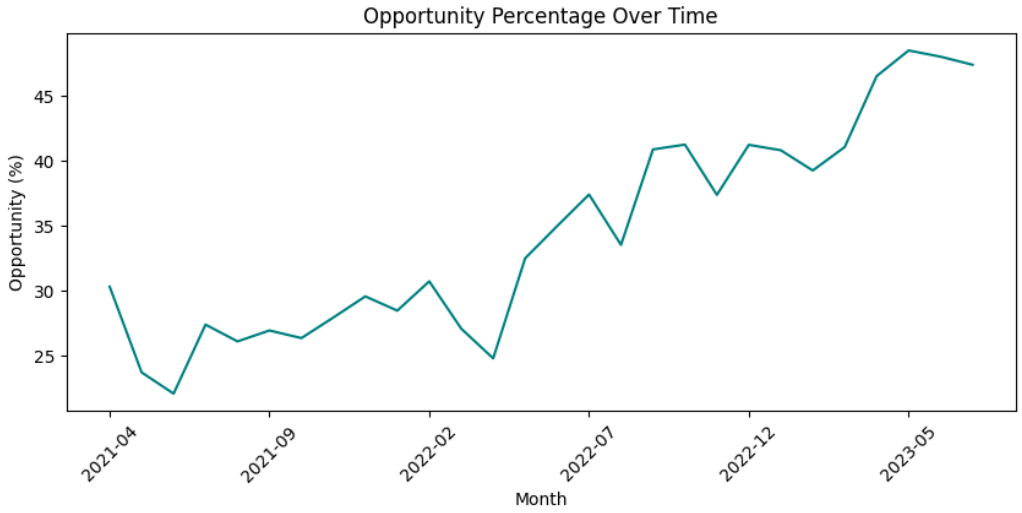



Figure 3: Monthly percentage of Opportunity observations between April 2021 and July 2023.

**Interpretation.** The Opportunity percentage generally increased over time. It ranged between approximately 22% and 30% during much of 2021 before rising to nearly 48% in 2023. This variation indicates temporal target drift, meaning that the distribution of growth opportunities changed between market periods. Therefore, chronological splitting is more appropriate than random splitting for evaluating the model.

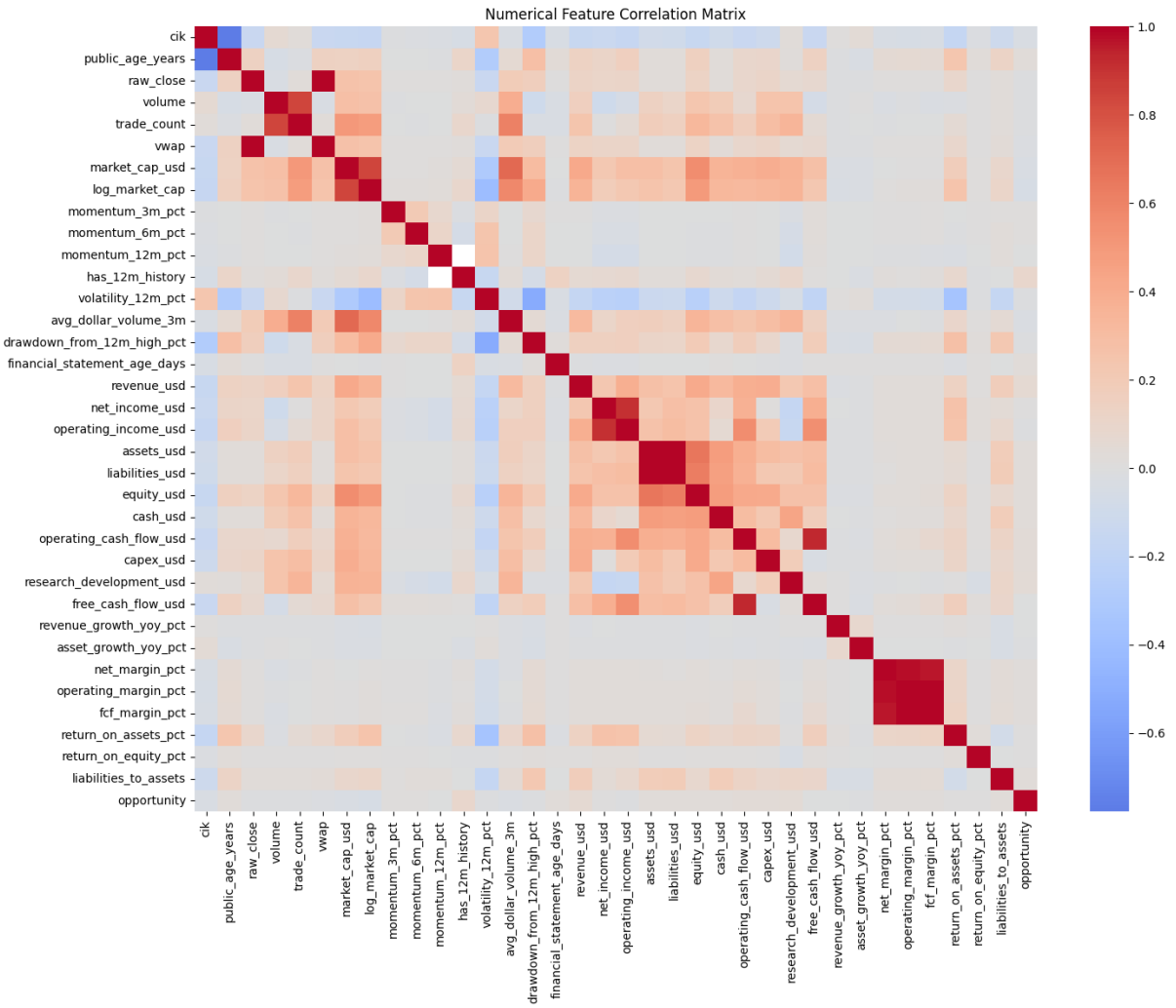



Figure 4: Pearson correlation matrix for the numerical variables.

**Interpretation.** Strong positive correlations appear between related financial variables, particularly assets, liabilities, revenue, income, and cash flow. The profitability margins are also highly related to one another. However, most features show weak individual linear relationships with the Opportunity target. The features were retained because tree-based models can identify nonlinear relationships and interactions that Pearson correlation does not capture.

The `cik` variable was not interpreted or used as a predictive feature because it is only an SEC company identifier.

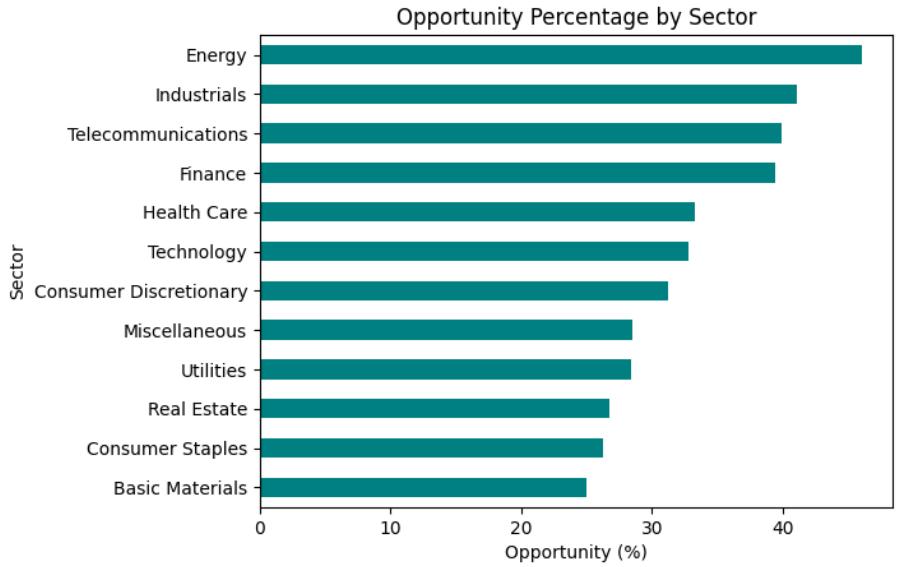



Figure 5: Percentage of Opportunity observations within each business sector.

**Interpretation.** The percentage of Opportunity observations differs across sectors. Energy recorded the highest rate at approximately 47%, followed by Industrials, Telecommunications, and Finance at approximately 40%. Basic Materials recorded the lowest rate at approximately 25%. These differences support using sector as a predictive feature. However, they represent statistical associations and do not prove that belonging to a particular sector causes future growth.

## 5.5 Data Splitting

Two temporal evaluation strategies were used. The chronological 80/10/10 split was used as the primary long-term evaluation strategy, while a blocked three-month/one-month split was used as an additional XGBoost experiment.

### 5.5.1 Chronological 80/10/10 Split

The observations were sorted using `observation_date`. The earliest 80% of unique dates were assigned to training, the following 10% to validation, and the latest 10% to testing. Complete dates were kept within a single set to avoid placing observations from the same date in different dataset.

Table 2: Chronological dataset split

| Set | Rows | Opportunity Rate |
|---|---|---|
| Training | 793,277 | 24.54% |
| Validation | 101,028 | 35.33% |
| Test | 105,695 | 39.61% |

The Opportunity rate increased from 24.54% in training to 39.61% in testing. This difference indicates temporal target drift: the distribution of future growth opportunities changed across market periods. The chronological split therefore provides a realistic and difficult evaluation than a random split.

### 5.5.2 Three-Month/One-Month Experimental Split

A second temporal experiment was created for XGBoost. The model was trained on three consecutive months and evaluated on the following month. This process was repeated across seven non-overlapping blocks between April 2021 and July 2023.

Table 3: Opportunity rates across three-month/one-month temporal folds

| Fold | Training Months | Test Month | Test Opportunity Rate |
|---|---|---|---|
| 1 | Apr–Jun 2021 | Jul 2021 | 20.72% |
| 2 | Aug–Oct 2021 | Nov 2021 | 20.70% |
| 3 | Dec 2021–Feb 2022 | Mar 2022 | 20.70% |
| 4 | Apr–Jun 2022 | Jul 2022 | 29.60% |
| 5 | Aug–Oct 2022 | Nov 2022 | 29.89% |
| 6 | Dec 2022–Feb 2023 | Mar 2023 | 33.32% |
| 7 | Apr–Jun 2023 | Jul 2023 | 39.11% |

This experiment measure short-period generalization across adjacent market periods. However, the same companies can appear in multiple blocks, and their three-year target periods can overlap. Therefore, this experiment is reported separately and is not treated as directly comparable with the chronological 80/10/10 evalution.

## 5.6 Data Preprocessing

### 5.6.1 Missing-Value Treatment

Missing categorical values were replaced with `Unknown`. For the chronological models, missing numerical values were replaced using medians calculated from chronological training set. The same training medians were then applied to validation and test data.

### 5.6.2 Feature Engineering

Five additional financial ratios were created:

- `revenue_to_marketcap`: revenue divided by market capitalization.
- `earnings_yield`: net income divided by market capitalization.
- `fcf_yield`: free cash flow divided by market capitalization.
- `cash_to_assets`: cash divided by total assets.
- `rd_intensity`: research and development expenditure divided by the absolute value of revenue.

These ratios allow the models to compare financial performance across companies of different sizes.

### 5.6.3 CatBoost Representation

CatBoost received 26 input features: three categorical features and 23 numerical features. The categorical columns `exchange`, `sector`, and `industry` were passed directly to CatBoost. One-hot encoding and feature scaling were not required because CatBoost can process categorical variables natively and tree-based models are not sensitive to feature scale.

### 5.6.4 cuML Stacking Representation

For the cuML stacking experiment, the three categorical variables were converted using one-hot encoding this increased feature matrix from 26 original feature to 180 numerical columns. PCA was then applied to reduce redundancy and dimensionality. As shown in Figure 6, 80 principal component preserved 95% of the total variance and were used as inputs to the cuML stacking model.

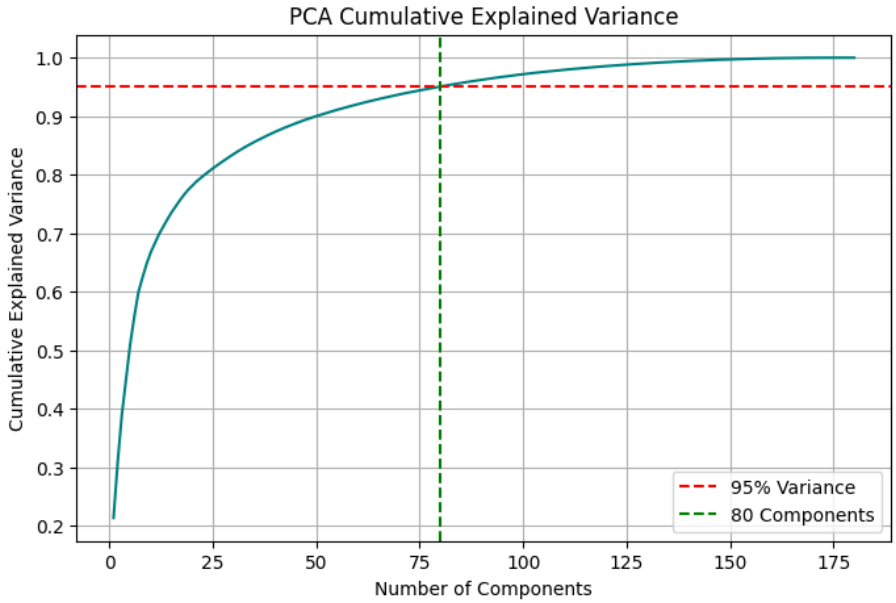



Figure 6: PCA cumulative explained variance. Eighty components preserved 95% of the variance in the encoded features.

### 5.6.5 XGBoost Fold Preprocessing

XGBoost preprocessing was performed separately within every three-month/one-month block. Categorical variables were one-hot encoded using training months of that block, and the following test month was aligned to the training columns.

Missing categories were represented explicitly during encoding. Remaining numerical missing values were processed natively by XGBoost. Scaling was not applied because XGBoost is a tree-based model.

### 5.6.6 Preprocessing Performance

GPU preprocessing required 10.38 seconds, reached an average GPU utilization of 18.17%, and used peak of 4,998.19 MB of GPU memory. The relatively low average utilzation is

expected because encoding, median calculation, and memory transfer do not continuously occupy all GPU computational units.

# 6 Methodology and Training

## 6.1 Models Considered

Three GPU-based classification approaches were evaluated:

- **cuML Stacking:** Combines Logistic Regression, Random Forest, and Gaussian Naive Bayes predictions using a Logistic Regression meta-model.
- **CatBoost Classifier:** A tree-based model that processes categorical features directly.
- **XGBoost Classifier:** A gradient-boosting model designed for accurate and efficient classification on large tabular datasets.

A majority-class baseline was also used. it classified every observation as *Not Opportunity* and provided a simple reference for evaluating trained models.

## 6.2 Training Setup and GPU Environment

Table 4: Computing environment

| **Component** | **Specification** |
|---|---|
| Platform | Google Colab |
| GPU | NVIDIA T4 |
| GPU memory | 15 GB |
| Data processing | RAPIDS cuDF |
| GPU machine learning | cuML / XGBoost / CatBoost |
| Dataset format | Parquet |

## 6.3 Hyperparameter Selection

A controlled set of hyperparameters was selected for each model rather than performing an exhaustive search. CatBoost used 1,000 iterations, depth 8,a learning rate of 0.05, and early stopping after 50 rounds without validation improvement. XGBoost used 1,000 estimators, depth 8, a learning rate of 0.05, row and feature subsampling of 0.8.

The chronological validation set was used to monitor CatBoost and select its best iteration. XGBoost was evaluated separately using the three-month/one-month temporal folds.

## 6.4 Baseline

The chronological baseline predicted every test observation as *Not Opportunity*, which was the majority class. It achieved 60.39% Accuracy and 50% Balanced Accuracy, but

its recall and F1-score for the Opportunity class were both zero. Therefore , the trained models were evaluated mainly using Balanced Accuracy, Recall, F1-score, and AUC rather than Accuracy alone.

# 7 Evaluation and Results

## 7.1 Evaluation Metrics

The models were evaluated using the following classification metrics:

- **Accuracy:** Percentage of all predictions that were correct.
- **Balanced Accuracy:** Average accuracy across the two classes.
- **Precision:** Percentage of predicted opportunities that were correct.
- **Recall:** Percentage of actual opportunities detected by the model.
- **F1-score:** Balance between Precision and Recall.
- **AUC:** Ability to rank Opportunity observations above Not Opportunity observations.

Balanced Accuracy, F1-score, and AUC were emphasized because the target classes were moderately imbalanced.

## 7.2 Results Table

Table 5: Classification performance of the evaluated models

| Model | Split | Accuracy | Balanced Acc. | Precision | Recall | F1 | AUC |
|---|---|---|---|---|---|---|---|
| cuML Stacking with PCA | Chronological 80/10/10 | 0.6882 | **0.6667** | 0.6164 | **0.5635** | **0.5888** | 0.7310 |
| CatBoost | Chronological 80/10/10 | 0.6978 | 0.6568 | 0.6740 | 0.4594 | 0.5464 | 0.7402 |
| XGBoost | Three Months / One Month | **0.9370** | **0.9090** | **0.9124** | **0.8481** | **0.8791** | **0.9623** |

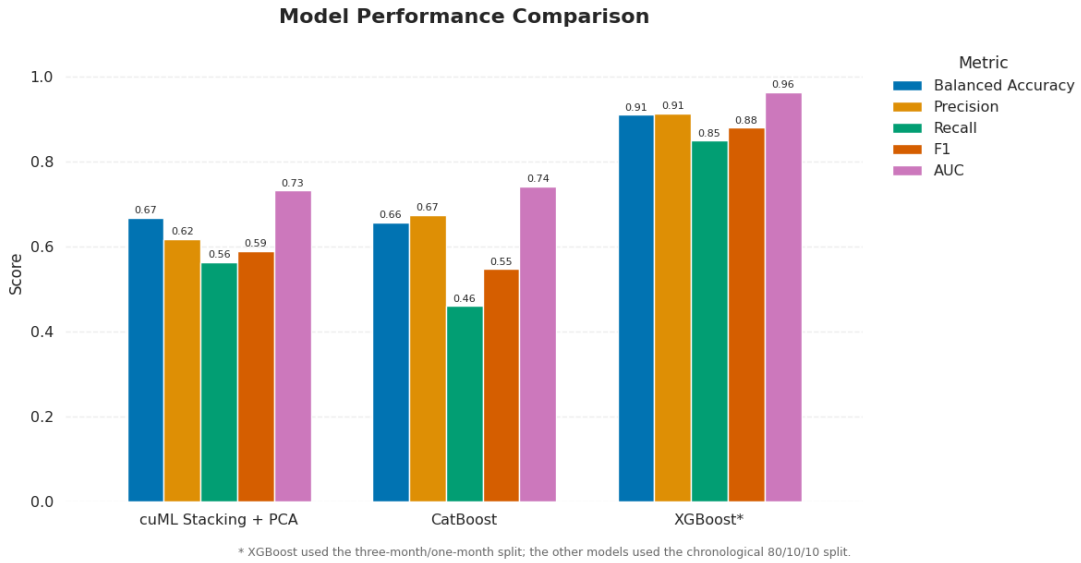



Figure 7: Classification performance of the three evaluated model configurations. XGBoost used a different temporal splitting strategy.

Among the models evaluated on the same chronological split, neither model was superior across every metric. CatBoost achieved higher accuracy, Precision, and AUC, while cuML Stacking with PCA achieved higher Balanced Accuracy, Recall and F1-score.

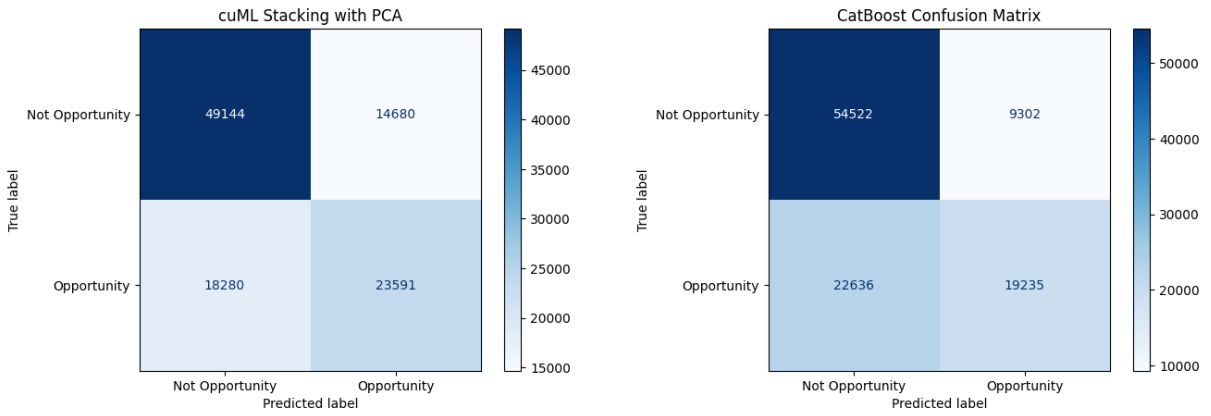
The cuML model was better at detecting actual Opportunity observations, wheras Catboost produced more reliable positive predictions but missed more opportunities. Therefore, cuML Stacking with PCA may be more suitable when missing a potential investment opportunity is considered costly.

XGBoost achieved the strongest experimental results, but it used a different temporal splitting strategy and cannot be compared directly with the two chronological models.

## 7.3 Error Analysis

CatBoost achieved Higher Precision than Recall. This means that its Opportunity predictions were relatively reliable, but it failed to detect many actual opportunities. The cuML Stacking model detected slightly more opportunities, but it produced more false positive and had a lower AUC.

XGBoost produced the best balance between Precision and Recall. However, the same companies and overlapping three-year target periods may appear across its temporal folds. Therefore, its performance may be more optimistic than the chronological evaluation.




(a) cuML Stacking with PCA (b) CatBoost

Figure 8: Confusion matrices for cuML Stacking with PCA and CatBoost on the same chronological test set.

Figure 8 shows that cuML Stacking with PCA detected more actual Opportunity observations, resulting in higher Recall. CatBoost produced fewer false-positive predictions, resulting in higher Precision.

## 7.4 Feature Importance and Interpretation

XGBoost feature importance was averaged across the seven temporal folds. The results identify the variables that the models used most frequently when making predictions.

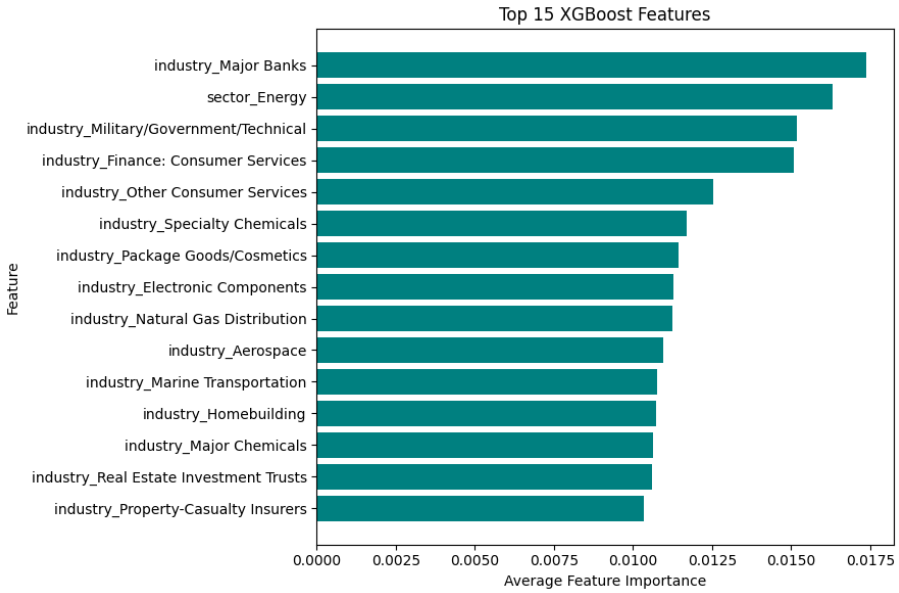



Figure 9: Top 15 features used by XGBoost across the temporal folds.

The feature-importance results show that XGBoost relied on a combination of market, financial, and company characteristic variables. This suggests that future growth opportunities cannot be explained by one factor alone, but by interactions between several company indicators.

# 8 GPU Performance Analysis

GPU performance was evaluated using average GPU utilization and peak GPU memory during preprocessing and model training.

## 8.1 GPU Utilization and Memory

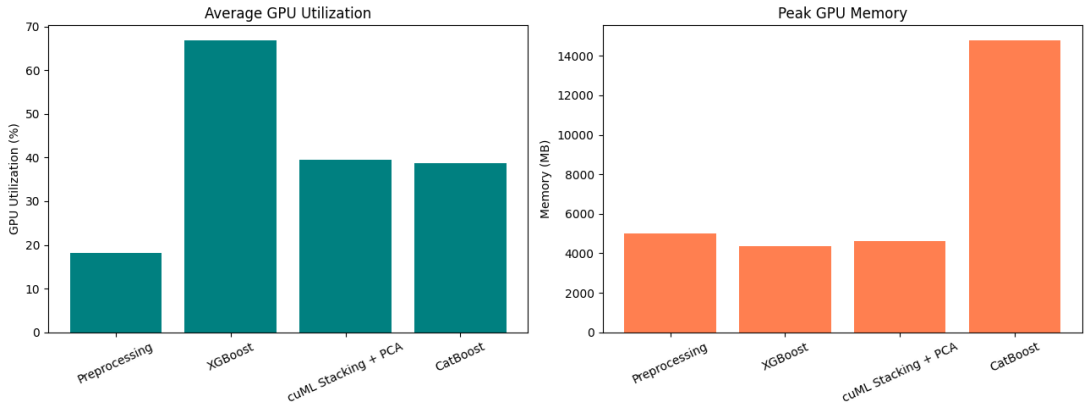



Figure 10: Average GPU utilization and peak GPU memory during preprocessing and model training.

As shown in Figure 10, XGBoost recorded the highest average GPU utilization at 66.89%. The cuML Stacking model with PCA and CatBoost recorded 39.47% and 38.63%, respectively. Preprocessing had the lowest utilization at 18.17% because data preparation does not require continues GPU computation.

CatBoost was the most memory-intensive model, reaching 14,802 MB and using almost all avaliable memory on the NVIDIA T4 GPU. XGBoost recorded the lowest model-training memory at 4,358 MB, followed by cuML Stacking with PCA at 4,620 MB.

Overall, XGBoost used the GPU most actively while maintaining the lowest model memory consumption. The cuML Stacking model with PCA required moderates GPU memory, whereas CatBoost had the highest memory cost.

## 8.2 Training and Inference Speed

Table 6: GPU resource usage by preprocessing and model

| **Phase** | **Average GPU (%)** | **Peak GPU Memory (MB)** |
|---|---|---|
| Preprocessing | 18.17 | 4,998.19 |
| XGBoost | 66.89 | 4,358.19 |
| cuML Stacking + PCA | 39.47 | 4,620.19 |
| CatBoost | 38.63 | 14,802.19 |

# 9 Challenges

- **Missing financial values:** Several financial features contained many missing values. Missing numerical values were filled using training-set medians, while missing categorical values were represented as `Unknown`.
- **Temporal target drift:** The Opportunity rate increased from 24.54% in the training set to 39.61% in test set. Chronological splitting was used to evaluate the models under this change instead of hiding it through random splitting.
- **GPU memory limitations:** CatBoost reached approximately 14,802 MB of GPU memory, close to the limit of NVIDIA T4. Parquet storage, and PCA dimensionality reduction helped control memory usage.
- **Model comparison:** XGBoost used three-month/one-month folds, while CatBoost and cuML used the chronological 80/10/10 split. Therefore, XGBoost results were reported as a separate experiment rather than a direct comparison.

# 10 Discussion

The results show that historical financial and market information contains useful patterns for identifying companies with future growth potential. Among models evaluated on the same chronological split, CatBoost produced more precise Opportunity predictions, while cuML Stacking with PCA detected a larger proportion of actual opportunities. XGBoost produced the strongest experimental results but used a different temporal evaluation strategy. The main limitations are temporal market drift, overlapping three-year target periods, and missing financial information.

GrowthLens should be treated as a screening and decision-support tool rather than a source of guaranteed investment advice. False positive could lead to poor investment decision, while false negative could hide valuable opportunities. Model predictions should therefore combined with current financial reports, risk analysis, company valuation, and hyman review

# 11 Future Work

- Add point-in-time NLP features extracted from SEC 10-K reports using FinBERT.
- Build semantic search over company reports for evidence-based explanations.
- Develop a RAG assistant combining predictions with retrieved filing evidence.
- Test the system on additional countries and newer company observations.

# 12 References

[1] U.S. Securities and Exchange Commission, "EDGAR Company Filings." https://www.sec.gov/edgar

[2] Alpaca Markets, "Historical Market Data API." https://alpaca.markets/data

[3] NVIDIA, "RAPIDS: GPU-Accelerated Data Science." https://rapids.ai/

[4] NVIDIA, "RAPIDS cuML Documentation." https://docs.rapids.ai/api/cuml/stable/

[5] XGBoost Developers, "XGBoost Documentation." https://xgboost.readthedocs.io/

[6] CatBoost Developers, "CatBoost Documentation." https://catboost.ai/